

Exercise 1: Inventory Management System

Scenario:

You are developing an inventory management system for a warehouse. Efficient data storage and retrieval are crucial.

Suitable Data Structure Usage: HashMap

Why?

- Every product has a unique productid
- ProductId can be used as the key.
- Searching, updating, and deleting become almost instantaneous.

Main.java

```
public class Main {  
    public static void main(String[] args) {  
        Inventory inventory = new Inventory();  
  
        inventory.addProduct(new Product(101, "Laptop", 20, 65000));  
        inventory.addProduct(new Product(102, "Mouse", 50, 500));  
        inventory.addProduct(new Product(103, "Keyboard", 40, 1200));  
  
        System.out.println();  
  
        inventory.displayProducts();  
  
        System.out.println();  
  
        inventory.updateProduct(102, 75, 550);  
  
        System.out.println();  
  
        inventory.deleteProduct(103);  
  
        System.out.println();  
  
        inventory.displayProducts();  
    }  
}
```

Product.java

```
public class Product {

    private int productId;
    private String productName;
    private int quantity;
    private double price;

    public Product(int productId, String productName, int quantity, double price) {
        this.productId = productId;
        this.productName = productName;
        this.quantity = quantity;
        this.price = price;
    }

    public int getProductId() {
        return productId;
    }

    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    @Override
    public String toString() {
        return "Product ID: " + productId +
            ", Name: " + productName +
            ", Quantity: " + quantity +
            ", Price: ₹" + price;
    }
}
```

Inventory.java

```
import java.util.HashMap;

public class Inventory {

    private HashMap<Integer, Product> inventory = new HashMap<>();

    // Add Product
    public void addProduct(Product product) {
        inventory.put(product.getProductId(), product);
        System.out.println("Product Added Successfully.");
    }

    // Update Product
    public void updateProduct(int id, int quantity, double price) {

        Product product = inventory.get(id);
```

```
        if (product != null) {
            product.setQuantity(quantity);
            product.setPrice(price);
            System.out.println("Product Updated.");
        } else {
            System.out.println("Product Not Found.");
        }
    }

    // Delete Product
    public void deleteProduct(int id) {

        if (inventory.remove(id) != null)
            System.out.println("Product Deleted.");
        else
            System.out.println("Product Not Found.");
    }

    // Display Products
    public void displayProducts() {

        for (Product product : inventory.values()) {
            System.out.println(product);
        }
    }
}
```

Time Complexity Analysis:

Operation	Hashmap
Add	O(1)
Search	O(1)
Update	O(1)
Delete	O(1)

Output Screenshot:

```
"C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.2\lib\idea_rt.jar=57261" -Dfile.encoding=UTF-8
Product Added Successfully.
Product Added Successfully.
Product Added Successfully.

Product ID: 101, Name: Laptop, Quantity: 20, Price: ₹65000.0
Product ID: 102, Name: Mouse, Quantity: 50, Price: ₹500.0
Product ID: 103, Name: Keyboard, Quantity: 40, Price: ₹1200.0

Product Updated.

Product Deleted.

Product ID: 101, Name: Laptop, Quantity: 20, Price: ₹65000.0
Product ID: 102, Name: Mouse, Quantity: 75, Price: ₹550.0

Process finished with exit code 0
```

Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

What is Big O Notation?

Big O Notation is a mathematical way of describing the efficiency of an algorithm in terms of time and space as the input size (n) increases.

It helps us:

- Compare different algorithms.
- Predict performance for large datasets.
- Choose the most efficient algorithm.

Big O	Performance	Example
$O(1)$	Constant	Accessing an array element
$O(\log n)$	Very Fast	Binary Search
$O(n)$	Linear	Linear Search
$O(n \log n)$	Efficient	Merge Sort, Quick Sort (Average)
$O(n^2)$	Slow	Bubble Sort

Best, Average, and Worst Cases of Search Operations

Linear Search

- **Best Case: $O(1)$**
 - Element is found at the first position.
- **Average Case: $O(n)$**
 - Element is found somewhere in the middle.
- **Worst Case: $O(n)$**
 - Element is at the last position or not present.

Code:

Main.java

```
import java.util.Arrays;
import java.util.Comparator;

public class Main {

    public static void main(String[] args) {

        Product[] products = {
            new Product(101, "Laptop", "Electronics"),
            new Product(102, "Mouse", "Electronics"),
            new Product(103, "Keyboard", "Electronics"),
            new Product(104, "Shoes", "Fashion"),
            new Product(105, "Watch", "Accessories")
        };

        // Linear Search
        Product result1 = SearchOperations.linearSearch(products,
"Keyboard");

        if (result1 != null)
            System.out.println("Linear Search Found: " + result1);
        else
            System.out.println("Product Not Found");

        // Sort array before Binary Search
        Arrays.sort(products, Comparator.comparing(p -> p.productName));

        Product result2 = SearchOperations.binarySearch(products,
"Keyboard");

        if (result2 != null)
            System.out.println("Binary Search Found: " + result2);
        else
            System.out.println("Product Not Found");
    }
}
```

Product.java

```
public class Product {

    int productId;
    String productName;
    String category;

    public Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    @Override
```

```

    public String toString() {
        return productId + " " + productName + " " + category;
    }
}

```

SearchOperations.java

```

public class SearchOperations {

    // Linear Search
    public static Product linearSearch(Product[] products, String name) {

        for (Product product : products) {
            if (product.productName.equalsIgnoreCase(name)) {
                return product;
            }
        }

        return null;
    }

    // Binary Search
    public static Product binarySearch(Product[] products, String name) {

        int low = 0;
        int high = products.length - 1;

        while (low <= high) {

            int mid = (low + high) / 2;

            int comparison =
products[mid].productName.compareToIgnoreCase(name);

            if (comparison == 0) {
                return products[mid];
            }

            if (comparison < 0) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        return null;
    }
}

```

Time Complexity Analysis:

Algorithm	Best Case	Average Case	Worst Case
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)

Output:

```
"C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.2\lib\idea_rt.jar=64092" -Dfile.encoding=UTF-8
Linear Search Found: 103 Keyboard Electronics
Binary Search Found: 103 Keyboard Electronics

Process finished with exit code 0
```

Exercise 3: Sorting Customer Orders

Scenario:

You are tasked with sorting customer orders by their total price on an e-commerce platform. This helps in prioritizing high-value orders.

Code:

Main.java

```
public class Main {

    public static void main(String[] args) {

        Order[] orders = {
            new Order(101, "Alice", 4500),
            new Order(102, "Bob", 1200),
            new Order(103, "Charlie", 7800),
            new Order(104, "David", 2500)
        };

        System.out.println("Bubble Sort:");

        SortingOperations.bubbleSort(orders);

        for (Order order : orders)
            System.out.println(order);

        System.out.println("\nQuick Sort:");

        Order[] orders2 = {
            new Order(101, "Alice", 4500),
            new Order(102, "Bob", 1200),
            new Order(103, "Charlie", 7800),
            new Order(104, "David", 2500)
        };

        SortingOperations.quickSort(orders2, 0, orders2.length - 1);

        for (Order order : orders2)
            System.out.println(order);
    }
}
```

```
}  
}
```

Order.java

```
public class Order {  
  
    int orderId;  
    String customerName;  
    double totalPrice;  
  
    public Order(int orderId, String customerName, double totalPrice) {  
        this.orderId = orderId;  
        this.customerName = customerName;  
        this.totalPrice = totalPrice;  
    }  
  
    @Override  
    public String toString() {  
        return orderId + " " + customerName + " ₹" + totalPrice;  
    }  
}
```

SortingOperations.java

```
public class SortingOperations {  
  
    // Bubble Sort  
    public static void bubbleSort(Order[] orders) {  
  
        int n = orders.length;  
  
        for (int i = 0; i < n - 1; i++) {  
  
            for (int j = 0; j < n - i - 1; j++) {  
  
                if (orders[j].totalPrice > orders[j + 1].totalPrice) {  
  
                    Order temp = orders[j];  
                    orders[j] = orders[j + 1];  
                    orders[j + 1] = temp;  
  
                }  
  
            }  
  
        }  
  
    }  
  
    // Quick Sort  
    public static void quickSort(Order[] orders, int low, int high) {  
  
        if (low < high) {
```



```

        int pivotIndex = partition(orders, low, high);

        quickSort(orders, low, pivotIndex - 1);
        quickSort(orders, pivotIndex + 1, high);
    }
}

private static int partition(Order[] orders, int low, int high) {

    double pivot = orders[high].totalPrice;

    int i = low - 1;

    for (int j = low; j < high; j++) {

        if (orders[j].totalPrice < pivot) {

            i++;

            Order temp = orders[i];
            orders[i] = orders[j];
            orders[j] = temp;
        }
    }

    Order temp = orders[i + 1];
    orders[i + 1] = orders[high];
    orders[high] = temp;

    return i + 1;
}
}

```

Time Complexity Analysis:

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

Output:

```

"C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.2\lib\idea_rt.jar=59094" -Dfile.encoding=UTF-8
Bubble Sort:
102 Bob *1200.0
104 David *2500.0
101 Alice *4500.0
103 Charlie *7800.0

Quick Sort:
102 Bob *1200.0
104 David *2500.0
101 Alice *4500.0
103 Charlie *7800.0

Process finished with exit code 0

```

Exercise 4: Employee Management System

Scenario:

You are developing an employee management system for a company. Efficiently managing employee records is crucial.

Code:

Main.java

```
public class Main {

    public static void main(String[] args) {

        EmployeeManagement management = new EmployeeManagement(5);

        management.addEmployee(new Employee(101, "Alice", "Developer",
60000));
        management.addEmployee(new Employee(102, "Bob", "Tester", 45000));
        management.addEmployee(new Employee(103, "Charlie", "Manager",
90000));

        System.out.println("\nEmployee List:");
        management.displayEmployees();

        System.out.println("\nSearching Employee:");
        Employee emp = management.searchEmployee(102);

        if (emp != null)
            System.out.println(emp);
        else
            System.out.println("Employee Not Found.");

        System.out.println("\nDeleting Employee:");
        management.deleteEmployee(102);

        System.out.println("\nUpdated Employee List:");
        management.displayEmployees();
    }
}
```

Employee.java

```
public class Employee {

    int employeeId;
    String name;
    String position;
    double salary;

    public Employee(int employeeId, String name, String position, double
salary) {
        this.employeeId = employeeId;
        this.name = name;
    }
}
```

```

        this.position = position;
        this.salary = salary;
    }

    @Override
    public String toString() {
        return employeeId + " " + name + " " + position + " ₹" + salary;
    }
}

```

EmployeeManagement.java

```

public class EmployeeManagement {

    private Employee[] employees;
    private int count;

    public EmployeeManagement(int size) {
        employees = new Employee[size];
        count = 0;
    }

    // Add Employee
    public void addEmployee(Employee employee) {

        if (count < employees.length) {
            employees[count++] = employee;
            System.out.println("Employee Added.");
        } else {
            System.out.println("Array is Full.");
        }
    }

    // Search Employee
    public Employee searchEmployee(int id) {

        for (int i = 0; i < count; i++) {

            if (employees[i].employeeId == id)
                return employees[i];
        }

        return null;
    }

    // Traverse Employees
    public void displayEmployees() {

        for (int i = 0; i < count; i++) {
            System.out.println(employees[i]);
        }
    }

    // Delete Employee
    public void deleteEmployee(int id) {

```

```

        int index = -1;

        for (int i = 0; i < count; i++) {

            if (employees[i].employeeId == id) {
                index = i;
                break;
            }
        }

        if (index == -1) {
            System.out.println("Employee Not Found.");
            return;
        }

        for (int i = index; i < count - 1; i++) {
            employees[i] = employees[i + 1];
        }

        employees[count - 1] = null;
        count--;

        System.out.println("Employee Deleted.");
    }
}

```

Time Complexity Analysis

Operation	Time Complexity
Add	O(1)
Search	O(n)
Traverse	O(n)
Delete	O(n)

Output:

```

"C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.2\lib\idea_rt.jar=59313" -Dfile.encoding=UTF-8
Employee Added.
Employee Added.
Employee Added.

Employee List:
101 Alice Developer ₹60000.0
102 Bob Tester ₹45000.0
103 Charlie Manager ₹90000.0

Searching Employee:
102 Bob Tester ₹45000.0

Deleting Employee:
Employee Deleted.

Updated Employee List:
101 Alice Developer ₹60000.0
103 Charlie Manager ₹90000.0

Process finished with exit code 0

```

Exercise 5: Task Management System

Scenario:

You are developing a task management system where tasks need to be added, deleted, and traversed efficiently.

Code:

Main.java

```
public class Main {

    public static void main(String[] args) {

        TaskManagement taskList = new TaskManagement();

        taskList.addTask(101, "Design UI", "Pending");
        taskList.addTask(102, "Develop Backend", "In Progress");
        taskList.addTask(103, "Testing", "Pending");

        System.out.println("Task List:");
        taskList.displayTasks();

        System.out.println("\nSearching Task:");
        Task task = taskList.searchTask(102);

        if (task != null)
            System.out.println(task);
        else
            System.out.println("Task Not Found.");

        System.out.println("\nDeleting Task:");
        taskList.deleteTask(102);

        System.out.println("\nUpdated Task List:");
        taskList.displayTasks();
    }
}
```

Task.java

```
public class Task {

    int taskId;
    String taskName;
    String status;
    Task next;

    public Task(int taskId, String taskName, String status) {
        this.taskId = taskId;
        this.taskName = taskName;
        this.status = status;
        this.next = null;
    }
}
```

```

@Override
public String toString() {
    return taskId + " " + taskName + " " + status;
}
}

```

TaskManagement.java

```

public class TaskManagement {

    private Task head;

    // Add Task
    public void addTask(int id, String name, String status) {

        Task newTask = new Task(id, name, status);

        if (head == null) {
            head = newTask;
            return;
        }

        Task temp = head;

        while (temp.next != null) {
            temp = temp.next;
        }

        temp.next = newTask;
    }

    // Search Task
    public Task searchTask(int id) {

        Task temp = head;

        while (temp != null) {

            if (temp.taskId == id)
                return temp;

            temp = temp.next;
        }

        return null;
    }

    // Traverse Tasks
    public void displayTasks() {

        Task temp = head;

        while (temp != null) {
            System.out.println(temp);
            temp = temp.next;
        }
    }
}

```

```

    }
}

// Delete Task
public void deleteTask(int id) {

    if (head == null) {
        System.out.println("Task List is Empty.");
        return;
    }

    if (head.taskId == id) {
        head = head.next;
        System.out.println("Task Deleted.");
        return;
    }

    Task current = head;
    Task previous = null;

    while (current != null && current.taskId != id) {
        previous = current;
        current = current.next;
    }

    if (current == null) {
        System.out.println("Task Not Found.");
        return;
    }

    previous.next = current.next;

    System.out.println("Task Deleted.");
}
}

```

Time Complexity Analysis

Operation	Time Complexity
Add	O(n)
Search	O(n)
Traverse	O(n)
Delete	O(n)

Output:

```

"C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.2\lib\idea_rt.jar=57649" -Dfile.encoding=UTF-8
Task List:
101 Design UI Pending
102 Develop Backend In Progress
103 Testing Pending

Searching Task:
102 Develop Backend In Progress

Deleting Task:
Task Deleted.

Updated Task List:
101 Design UI Pending
103 Testing Pending

Process finished with exit code 0
|

```

Exercise 6: Library Management System

Scenario:

You are developing a library management system where users can search for books by title or author.

Code:

Main.java

```
import java.util.Arrays;
import java.util.Comparator;

public class Main {

    public static void main(String[] args) {

        Book[] books = {

            new Book(101, "Java", "James Gosling"),
            new Book(102, "Python", "Guido van Rossum"),
            new Book(103, "C++", "Bjarne Stroustrup"),
            new Book(104, "DBMS", "Korth")

        };

        // Linear Search
        Book result1 = Library.linearSearch(books, "Python");

        if (result1 != null)
            System.out.println("Linear Search Found: " + result1);
        else
            System.out.println("Book Not Found");

        // Sort before Binary Search
        Arrays.sort(books, Comparator.comparing(book -> book.title));

        // Binary Search
        Book result2 = Library.binarySearch(books, "Python");

        if (result2 != null)
            System.out.println("Binary Search Found: " + result2);
        else
            System.out.println("Book Not Found");

    }

}
```

Book.java

```
public class Book {

    int bookId;
    String title;
    String author;

    public Book(int bookId, String title, String author) {
```



```

        this.bookId = bookId;
        this.title = title;
        this.author = author;
    }

    @Override
    public String toString() {
        return bookId + " " + title + " " + author;
    }
}

```

Library.java

```

public class Library {

    // Linear Search
    public static Book linearSearch(Book[] books, String title) {

        for (Book book : books) {

            if (book.title.equalsIgnoreCase(title)) {
                return book;
            }

        }

        return null;
    }

    // Binary Search
    public static Book binarySearch(Book[] books, String title) {

        int low = 0;
        int high = books.length - 1;

        while (low <= high) {

            int mid = (low + high) / 2;

            int result = books[mid].title.compareToIgnoreCase(title);

            if (result == 0)
                return books[mid];

            if (result < 0)
                low = mid + 1;
            else
                high = mid - 1;
        }

        return null;
    }
}

```

Time Complexity Analysis:

Operation	Best Case	Average Case	Worst Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

Output:

```
"C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.2\lib\idea_rt.jar=63420" -Dfile.encoding=UTF-8
Linear Search Found: 102 Python Guido van Rossum
Binary Search Found: 102 Python Guido van Rossum

Process finished with exit code 0
```

Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Code:

Main.java

```
public class Main {

    public static void main(String[] args) {

        double currentValue = 10000;
        double growthRate = 0.10; // 10%
        int years = 3;

        double future = FinancialForecast.futureValue(currentValue,
growthRate, years);

        System.out.println("Future Value = ₹" + future);

    }

}
```

FinancialForecast.java

```
public class FinancialForecast {

    public static double futureValue(double currentValue, double growthRate,
int years) {
```

```
        // Base Case
        if (years == 0)
            return currentValue;

        // Recursive Case
        return futureValue(currentValue * (1 + growthRate), growthRate, years
- 1);
    }
}
```

Time Complexity:

Each recursive call processes one year.

If there are n years:

$$T(n) = T(n-1) + O(1)$$

Therefore:

- Time Complexity: $O(n)$
- Space Complexity: $O(n)$ (due to the recursive call stack)

Output:

```
"C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.2\lib\idea_rt.jar=57459" -Dfile.encoding=UTF
Future Value = ₹13310.000000000004

Process finished with exit code 0
```